



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS

**A PROJECT ON MACHINE UNLEARNING SDK AND API PLATFORM FOR
PRIVACY-COMPLIANT SYSTEM**

BY

DEBENDRA SHEKHAR GUPTA (46630)

DIPESH ACHARYA (46632)

JEEVAN KUMAR KUSHWAHA (46636)

PRIYANKA KUMARI MISHRA (46655)

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS

DHARAN, NEPAL

March 2026

A Machine Unlearning SDK and API Platform for Privacy-Compliant ML Systems

BY

DEBENDRA SHEKHAR GUPTA (46630)

DIPESH ACHARYA (46632)

JEEVAN KUMAR KUSHWAHA (46636)

PRIYANKA KUMARI MISHRA (46655)

Project Supervisor

Asst. Prof. Pukar Karki

A project submitted to the Department of Electronics and Computer Engineering in partial fulfillment
of the requirements for the Bachelor's Degree in Computer Engineering

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS, INSTITUTE OF ENGINEERING

TRIBHUVAN UNIVERSITY

DHARAN, NEPAL

March 2026

COPYRIGHT ©

The author has agreed that the Library, Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor(s) who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering in any use of the material of this project report. Copying, publication, or other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report, in whole or in part, should be addressed to:

Head

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering

Dharan, Sunsari

Nepal

DECLARATION

We declare that the work hereby submitted for the degree of Bachelor of Engineering in Computer Engineering at the Institute of Engineering, Purwanchal Campus entitled **“A MACHINE UNLEARNING SDK AND API PLATFORM FOR PRIVACY-COMPLIANT SYSTEM”** is our own work and has not been previously submitted to any university for any academic award.

We authorize the Institute of Engineering, Purwanchal Campus to lend this report to other institutions or individuals for the purpose of scholarly research.

DEBENDRA SHEKHAR GUPTA (46630)

DIPESH ACHARYA (46632)

JEEVAN KUMAR KUSHWAHA (46636)

PRIYANKA KUMARI MISHRA (46655)

March 2026

LETTER OF APPROVAL

ACKNOWLEDGEMENT

We wish to express our heartfelt gratitude to everyone who has contributed to the successful realization of this project.

We are immensely grateful to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering, Tribhuvan University, and the Head of Department, Asst. Prof. Manoj Kumar Guragain, for providing us with the essential resources, infrastructure, and opportunities to bring our ideas to life.

We would like to express our sincere gratitude to our respected project supervisor, Asst. Prof. Pukar Karki, for his invaluable guidance, encouragement, and continuous support throughout the course of this project. His expert advice, constructive feedback, and keen insights have been instrumental in shaping our work and ensuring its successful completion.

We also extend our appreciation to all the teachers and staff of the department for their guidance, mentorship, and assistance, which have been instrumental in supporting our academic and project journey. We are equally thankful to our colleagues and peers for their constructive feedback, collaborative brainstorming sessions, and moral support, which kept us motivated and inspired during the more challenging moments of this journey.

Lastly, we would like to express our deepest gratitude to our families and friends for their steadfast support, patience, and encouragement, which were invaluable throughout this endeavor.

This project stands as a testament to the collective efforts and contributions of everyone mentioned above. We are truly thankful for their support and inspiration.

Thank You!

ABSTRACT

Privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) grant users the legal right to request deletion of their personal data from machine learning systems. Honoring such requests is technically non-trivial, as trained models implicitly encode the influence of every data point seen during training. Retraining from scratch on every deletion request is computationally prohibitive, particularly as dataset sizes and model complexity grow.

This project presents PyUnlearn, a practical machine unlearning SDK and API platform built on the Sharded, Isolated, Sliced, and Aggregated (SISA) framework. By partitioning training data into independently retrainable shards, the system localizes the impact of any deletion request to a single shard, avoiding full model retraining. The system is designed to be compatible with any scikit-learn estimator, including Linear Regression, Logistic Regression, Support Vector Machines, Random Forests, and Gradient Boosting models, making it broadly applicable across classical machine learning workflows.

The architecture incorporates persistent SQLite-based metadata storage for stable index-to-shard mappings, an append-only audit log for regulatory traceability, and idempotent unlearning semantics to ensure operational safety under repeated deletion requests. An optional non-blocking background synchronization layer enables real-time monitoring through a Django backend and React-based dashboard without disrupting core training or unlearning operations. The system is packaged as a reusable SDK with configuration-driven deployment supporting both local and cloud-enabled environments.

Performance benchmarks on datasets ranging from 5,000 to 50,000 samples demonstrate significant speedup factors over full retraining, with accuracy retention maintained across varying shard configurations. Remaining work includes extended support for deep learning architectures, additional dataset validation, and completion of the production deployment pipeline.

Keywords: *Machine Unlearning, SISA Framework, Right to be Forgotten, GDPR, Privacy-Compliant Machine Learning, Selective Data Deletion, Federated Unlearning, scikit-learn, SDK, Audit Logging, Sharded Training, Data Privacy*

TABLE OF CONTENTS

COPYRIGHT	iii
LETTER OF APPROVAL	v
ACKNOWLEDGEMENT	vi
ABSTRACT	vii
LIST OF FIGURES	xii
LIST OF ABBREVIATIONS	xiii
1 INTRODUCTION	1
1.1 Background	1
1.2 Gap Identification	2
1.3 Motivation	3
1.4 Objectives	3
1.5 Scope and Limitations	4
2 RELATED THEORY	5
2.1 Machine Unlearning	5
2.2 Taxonomy of Unlearning Approaches	6
2.2.1 Exact Unlearning	6
2.2.2 Approximate Unlearning	6
2.2.3 Certified Data Removal	7
2.3 SISA Framework for Efficient Unlearning	7
2.3.1 Sharding	7
2.3.2 Slicing	8

2.3.3	Aggregation	8
2.3.4	Unlearning Time Analysis	9
2.4	Privacy Threat Models in Machine Learning	10
2.4.1	Membership Inference Attacks	10
2.4.2	Model Inversion Attacks	10
2.5	Model Architectures Supporting Machine Unlearning	11
2.5.1	Classical Machine Learning Models	11
2.5.2	Neural Network Architectures	11
2.5.3	Aggregation Strategies and Their Impact on Accuracy	12
3	LITERATURE REVIEW	14
4	METHODOLOGY	18
4.1	Project Development Method	18
4.1.1	Research	18
4.1.2	Strategize	19
4.1.3	Document Requirements	19
4.1.4	Develop	20
4.1.5	Integration and Testing	20
4.1.6	Deploy	20
4.1.7	Support and Maintenance	21
4.2	System Architecture	21
4.3	Sharded Training and Unlearning Framework	23
4.3.1	Data Partitioning Strategy	23
4.3.2	Training Pipeline	23

4.3.3	Unlearning Engine	24
4.4	Metadata Storage Design	25
4.4.1	Schema Design	25
4.4.2	Persistence and Portability	25
4.5	Monitoring Dashboard	26
5	RESULT AND DISCUSSION	27
5.1	System Implementation	27
5.1.1	SDK Core and Algorithm Support	27
5.1.2	Persistent History and Auditability	27
5.1.3	Configuration and Deployment	28
5.2	Monitoring Dashboard	29
5.3	Performance Evaluation	31
5.3.1	Experimental Setup	31
5.3.2	Impact of Shard Count on Unlearning Latency	31
5.3.3	Speedup Factor Over Full Retraining	32
5.3.4	Accuracy Retention After Unlearning	33
5.4	Challenges Faced	33
5.4.1	Hardware Constraints	33
5.4.2	Dataset Availability	34
5.4.3	Model-Specific Constraints	34
5.4.4	Operational Robustness	34
6	CONCLUSION AND FUTURE WORK	35
6.1	Conclusion	35

6.2	Limitations	36
6.2.1	Restricted to Classical Machine Learning Models	36
6.2.2	Fixed Sharding Strategy	36
6.2.3	Absence of Formal Privacy Certification	37
6.2.4	Scalability Constraints	37
6.2.5	Limited Dataset Diversity in Evaluation	37
6.3	Future Enhancements	38
6.3.1	Deep Learning Support	38
6.3.2	Adaptive Sharding	38
6.3.3	Formal Privacy Certification	38
6.3.4	Distributed and Cloud-Native Deployment	39
6.3.5	Extended Evaluation and Benchmarking	39
6.3.6	Federated Unlearning	39

LIST OF FIGURES

Figure 2.1: SISA Training Framework: Dataset partitioned into shards, each further divided into slices. On an unlearning request, only the affected shard is retrained from the last clean checkpoint.	9
Figure 2.2: Comparison of model architectures and their suitability for SISA-based unlearning: classical models (left) offer stable retraining, while deep networks (right) introduce optimization complexity.	12
Figure 4.1: Phases of the Agile Development Methodology adopted for PyUnlearn .	18
Figure 4.2: High-level system architecture of PyUnlearn showing the SDK core, meta-data layer, and monitoring dashboard	22
Figure 4.3: SISA-based training and unlearning pipeline showing data flow from partitioning through training to aggregated prediction	24
Figure 4.4: Configuration file setup showing API key and endpoint configuration for dashboard synchronization	26
Figure 5.1: YAML configuration file showing API key and endpoint setup for dashboard synchronization	29
Figure 5.2: Main dashboard interface showing model run history and unlearning event log	30
Figure 5.3: Dashboard view showing per-shard statistics and active sample counts after unlearning	30
Figure 5.4: Impact of shard count on unlearning latency and model accuracy in the SISA framework	32
Figure 5.5: Speedup factor of SISA unlearning compared to full retraining across dataset sizes from 5K to 50K samples	33

LIST OF ABBREVIATIONS

API	: Application Programming Interface
Colab	: Colaboratory
SDK	: Software Development Kit
GDPR	: General Data Protection Regulation
MLOps	: Machine Learning Operations
SISA	: Sharded, Isolated, Sliced and Aggregated
CCPA	: California Consumer Privacy Act
AI	: Artificial Intelligence
ML	: Machine Learning
CNN	: Convolutional Neural Network
MLP	: Multi-Layer Perceptron
CSV	: Comma-Separated Values
IEEE	: Institute of Electrical and Electronics Engineers
NeurIPS	: Neural Information Processing Systems
ICML	: International Conference on Machine Learning
arXiv	: arXiv e-print archive
UCI	: University of California, Irvine
CVPR	: Conference on Computer Vision and Pattern Recognition
CVF	: Computer Vision Foundation

CHAPTER 1

INTRODUCTION

1.1 Background

Machine learning models have become deeply integrated into everyday applications, from personalized content recommendations to critical tasks like medical diagnosis and financial fraud detection. These models learn patterns from vast amounts of user data, improving their accuracy and usefulness over time. However, this dependence on user data creates a fundamental tension with privacy rights. Regulations such as the General Data Protection Regulation (GDPR) in Europe and the California Consumer Privacy Act (CCPA) in the United States now establish clear legal requirements: users have the right to request deletion of their personal data from systems that process it. Beyond these two landmark regulations, similar legislation such as PIPEDA in Canada and the Personal Data Protection Act (PDPA) in various Asian jurisdictions further reinforce the global momentum toward data sovereignty and the right to be forgotten.

The technical challenge lies in how to honor such deletion requests when the data has already been incorporated into a trained machine learning model. Because ML models potentially memorize training data during the learning process, simply removing a record from a database does not eliminate its influence from a deployed model. This memorization effect is particularly pronounced in overparameterized models such as deep neural networks, which have sufficient capacity to encode individual training examples. As a result, a model trained on sensitive data may remain vulnerable to membership inference attacks and other privacy exploits even after the underlying data has been deleted from storage.

Traditional approaches require complete model retraining on the remaining dataset, a process that becomes prohibitively expensive as model complexity and dataset sizes grow. Retraining a large model could take hours or days, making it impossible to honor deletion requests within reasonable timeframes, especially when multiple deletion requests arrive frequently. Furthermore, in online learning settings where models are continuously updated with new data, the cost of repeated full retraining compounds significantly over time.

This gap between legal requirements and technical capability has given rise to the field of machine unlearning, which aims to efficiently remove the influence of specific data points from

trained models without full retraining. Unlearning guarantees that training on a point and subsequently unlearning it will produce the same distribution of models as not training on that point at all. This strong guarantee is what distinguishes machine unlearning from related privacy-preserving techniques such as differential privacy, which only bounds the contribution of individual points rather than eliminating it entirely.

1.2 Gap Identification

Despite growing research interest in machine unlearning, the practical adoption of these techniques remains limited. Academic papers propose sophisticated algorithms, but they often remain as theoretical prototypes or are tightly coupled to specific model architectures. Prior work has shown that exact unlearning is possible when the learning algorithm queries data in a fixed, predetermined order, as in the statistical query (SQ) learning setting. However, when the dataset is queried adaptively, convergence of such approaches is no longer guaranteed, and efficient algorithms for complex models such as deep neural networks do not exist in this setting.

Approximate unlearning methods, including influence functions and knowledge distillation on cleaned datasets, offer alternatives but introduce their own limitations. Influence functions require access to second-order gradient information, which is computationally expensive for large models. Knowledge distillation approaches depend on the availability of a suitable proxy dataset and do not provide formal guarantees of data removal. These constraints make them difficult to deploy reliably in production environments where regulatory compliance demands verifiable and auditable deletion.

Open-source libraries that do exist tend to focus on narrow use cases or require significant expertise to integrate into existing machine learning pipelines. For most developers and organizations, there is no straightforward, production-ready solution that allows them to build privacy-compliant systems with unlearning capabilities from day one. This project addresses that gap by developing PyUnlearn, a practical SDK and API platform that abstracts away the complexity of machine unlearning, making it accessible to developers working on real-world applications that must comply with privacy regulations.

1.3 Motivation

The motivation for this project stems from a simple but increasingly urgent question: as users entrust their data to machine learning systems, can they also trust that their presence can be removed when requested? Current practices often treat data deletion as a database operation, ignoring the fact that the model itself retains implicit knowledge of the deleted data. This disconnect between what users expect and what systems actually deliver creates legal risk for organizations and erodes user trust.

The SISA (Sharded, Isolated, Sliced, and Aggregated) training framework, introduced by Bourtoule et al., provides a structured and practical approach to this problem. By dividing training data into multiple disjoint shards and training models in isolation on each shard, the influence of any single data point is limited to the model trained on the shard containing that point. When an unlearning request arrives, only the affected shard needs to be retrained, dramatically reducing computational overhead compared to full retraining and making it feasible to honor deletion requests within practical timeframes.

Rather than treating privacy as an afterthought or a compliance checkbox, we believe that unlearning capabilities should be built directly into the machine learning development workflow. PyUnlearn represents an effort to make this possible, providing developers with tools that integrate naturally with existing scikit-learn-based workflows while ensuring that privacy guarantees are maintained throughout the model lifecycle. The system further extends the SISA framework with production-grade features including persistent metadata storage, audit logging, idempotent deletion semantics, and an optional real-time monitoring dashboard.

1.4 Objectives

The project aims to achieve the following objectives:

- Develop an efficient SISA-based selective unlearning SDK compatible with any scikit-learn estimator, enabling localized retraining without full model rebuilding.
- Ensure regulatory compliance by providing persistent audit logging, idempotent deletion semantics, and verifiable unlearning behavior aligned with GDPR and CCPA require-

ments.

- Deliver a production-ready platform with a Django and React monitoring dashboard, supporting both local and cloud-enabled deployments through configuration-driven setup.

1.5 Scope and Limitations

This project focuses on implementing a practical machine unlearning SDK based on the SISA framework, targeting tabular datasets and machine learning models compatible with the scikit-learn ecosystem. The system supports both local deployment and cloud-enabled monitoring through an optional dashboard built with Django and React. The SDK is designed to handle both individual and batch deletion requests, with persistent SQLite-based metadata ensuring stable shard mappings and full auditability across repeated executions. Configuration-driven deployment allows users to switch between local-only and cloud-synchronized modes without modifying their core machine learning code, making the system adaptable to a wide range of development and production environments.

While the approach demonstrates significant efficiency gains over full retraining, current limitations include the lack of support for deep learning architectures such as convolutional neural networks. The SISA framework achieves the largest improvements for stateful algorithms like stochastic gradient descent used in deep neural networks; however, adapting it to such architectures introduces additional complexity around gradient propagation and initialization strategies that are beyond the current scope. The system also relies on a fixed sharding strategy that may not adapt optimally to all dataset distributions, and the optimal shard count must currently be determined empirically rather than derived automatically from dataset characteristics. Furthermore, formal certification guarantees for the unlearning process are not provided, meaning the system offers practical rather than mathematically provable privacy assurances. These limitations represent well-defined directions for future research and engineering work beyond the scope of the current project.

CHAPTER 2

RELATED THEORY

2.1 Machine Unlearning

Machine unlearning refers to a class of methods that remove the influence of specific training records from a trained model while preserving utility on non-removed data. The motivation is both regulatory and operational: privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) establish the *Right to be Forgotten*, giving users the legal right to request deletion of their personal data. Beyond compliance, practical machine learning pipelines frequently require removal of mislabeled, outdated, or sensitive samples after a model has already been deployed.

Formally, let $\mathcal{D} = \{z_1, z_2, \dots, z_n\}$ denote a training dataset and let $\mathcal{A}(\mathcal{D})$ denote a learning algorithm that produces a model θ . Given a subset $\mathcal{D}_f \subset \mathcal{D}$ of data points to be forgotten, the goal of machine unlearning is to produce a model θ' such that:

$$\mathcal{A}(\mathcal{D} \setminus \mathcal{D}_f) \approx \mathcal{U}(\theta, \mathcal{D}_f, \mathcal{D})$$

where $\mathcal{U}$ is the unlearning algorithm. In the exact unlearning setting, this approximation becomes an equality in distribution, meaning the unlearned model is statistically indistinguishable from a model retrained from scratch on $\mathcal{D} \setminus \mathcal{D}_f$.

In conventional machine learning, deletion is typically handled by retraining the model from scratch on the remaining dataset. While this approach is conceptually simple and guarantees complete removal of the target data's influence, it becomes impractical as models grow in size and datasets expand. Retraining a large model could take hours or days, making it impossible to honor deletion requests within reasonable timeframes, especially when multiple deletion requests arrive frequently.

Beyond computational efficiency, a practical unlearning system must address two additional concerns. First, *verifiability* asks whether we can be confident that the target record no longer influences the model's predictions. Second, *auditability* requires the system to maintain a clear, tamper-resistant record of deletion requests and their outcomes. In real-world deployments, this

demands persistent bookkeeping structures that track active sample counts, shard assignments, and unlearning history, ensuring that model state remains consistent across repeated runs and system restarts.

2.2 Taxonomy of Unlearning Approaches

Machine unlearning methods can be broadly categorized into three families based on their underlying mechanism and the guarantees they provide.

2.2.1 Exact Unlearning

Exact unlearning methods guarantee that the resulting model is drawn from the same distribution as a model retrained from scratch on the remaining data. The SISA framework falls into this category. Exact unlearning is achievable when the training procedure can be decomposed into independent components, each of which can be retrained in isolation. The key condition is that the training data is partitioned such that each data point influences only one component of the final model.

2.2.2 Approximate Unlearning

Approximate unlearning methods aim to produce a model that is close to the exactly unlearned model, without the computational cost of full retraining. One prominent approach uses *influence functions* to estimate the effect of removing a training point on model parameters. Given a model with parameters θ trained on dataset $\mathcal{D}$, the influence of removing a point z on the parameters is approximated as:

$$\Delta\theta \approx -\frac{1}{n}H_{\theta}^{-1}\nabla_{\theta}\ell(\theta, z)$$

where $H_{\theta} = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta}^2 \ell(\theta, z_i)$ is the Hessian of the training loss and $\ell(\theta, z)$ is the loss on point z . While theoretically elegant, computing the inverse Hessian is $O(d^3)$ in the number of parameters d , making it computationally prohibitive for large models.

2.2.3 Certified Data Removal

Certified data removal, proposed by Guo et al., provides a formal privacy guarantee analogous to differential privacy. A mechanism $\mathcal{M}$ provides (ϵ, δ) -certified removal if for any dataset $\mathcal{D}$, any point $z \in \mathcal{D}$, and any set of outputs $\mathcal{S}$:

$$\Pr[\mathcal{M}(\mathcal{D}) \in \mathcal{S}] \leq e^\epsilon \Pr[\mathcal{M}(\mathcal{D} \setminus \{z\}) \in \mathcal{S}] + \delta$$

This guarantee ensures that an adversary cannot distinguish between a model trained on $\mathcal{D}$ and one from which z has been removed, up to the privacy budget (ϵ, δ) . However, achieving tight certified removal bounds typically requires gradient-based updates and is most effective for convex models.

2.3 SISA Framework for Efficient Unlearning

The Sharded, Isolated, Sliced, and Aggregated (SISA) framework, introduced by Bourtole et al., provides a structured approach to efficient machine unlearning by decomposing training into smaller, independently retrainable components. The core idea is straightforward: instead of training one monolithic model on the entire dataset, the dataset is divided into multiple independent *shards*. Each shard is trained separately, producing its own sub-model. When a prediction is needed, the outputs from all shard models are aggregated, typically through majority voting for classification or averaging for regression.

2.3.1 Sharding

Given a dataset $\mathcal{D}$ of n samples, the sharding step partitions $\mathcal{D}$ into S disjoint subsets:

$$\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_S, \quad \mathcal{D}_i \cap \mathcal{D}_j = \emptyset \quad \forall i \neq j$$

Each shard $\mathcal{D}_s$ contains approximately $\frac{n}{S}$ samples. The optimal number of shards S can be estimated based on the dataset dimensionality d , total sample count n , and a configurable target samples-per-shard parameter r :

$$S = \sqrt{\frac{d \cdot n}{r}}$$

This formula balances the trade-off between retraining efficiency (favoring more shards) and model accuracy (favoring fewer, larger shards). A separate model θ_s is trained independently on each shard $\mathcal{D}_s$.

2.3.2 Slicing

To further reduce retraining time, each shard $\mathcal{D}_s$ is divided into K time-ordered slices $\mathcal{D}_s^1, \mathcal{D}_s^2, \dots, \mathcal{D}_s^K$. The model is trained incrementally, and the parameter state θ_s^k is saved after processing each slice k . When a deletion request targets a point in slice k^* , retraining begins from the saved state $\theta_s^{k^*-1}$ rather than from random initialization:

$$\theta_s^{k^*}, \dots, \theta_s^K \leftarrow \text{Retrain}(\theta_s^{k^*-1}, \mathcal{D}_s^{k^*} \setminus \{z\}, \mathcal{D}_s^{k^*+1}, \dots, \mathcal{D}_s^K)$$

This slicing mechanism provides an additional reduction in retraining time at the cost of increased storage for intermediate checkpoints.

2.3.3 Aggregation

At inference time, predictions from all S shard models are combined. For classification tasks, majority voting is used:

$$\hat{y} = \arg \max_c \sum_{s=1}^S \mathbf{1}[\hat{y}_s = c]$$

For regression tasks, the final prediction is the average of shard predictions:

$$\hat{y} = \frac{1}{S} \sum_{s=1}^S \hat{y}_s$$

The key insight for unlearning is that any training record belongs to exactly one shard. Therefore, when a deletion request arrives, only the shard containing that record needs to be retrained,

leaving all other shards untouched. This localized retraining dramatically reduces computational overhead compared to full model retraining.

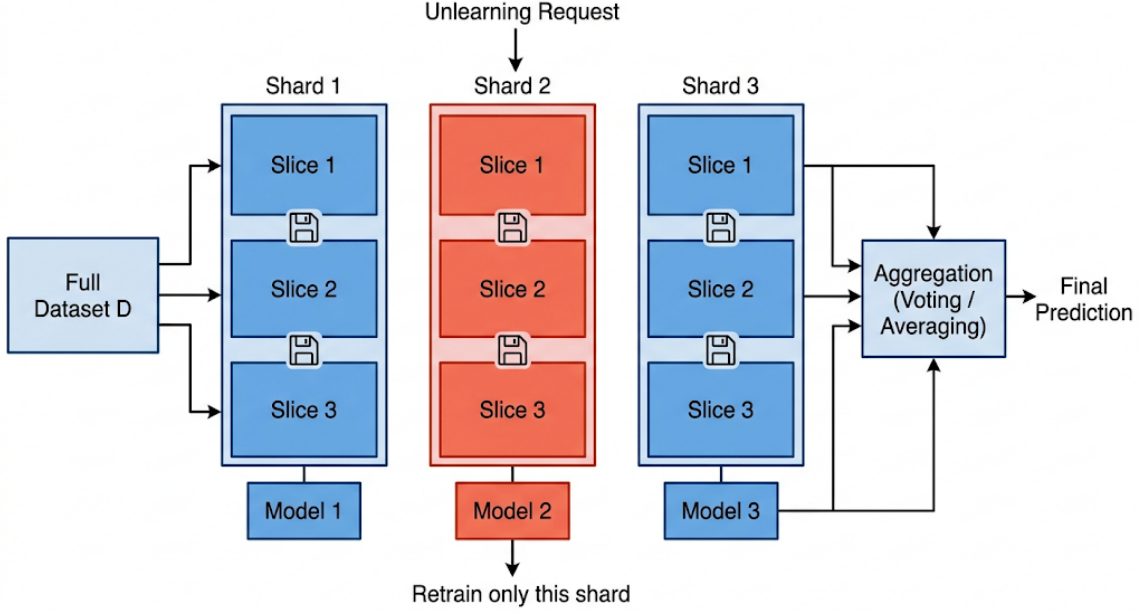


Figure 2.1: SISA Training Framework: Dataset partitioned into shards, each further divided into slices. On an unlearning request, only the affected shard is retrained from the last clean checkpoint.

2.3.4 Unlearning Time Analysis

Without SISA, the time to retrain a model after a single deletion request is T , the full training time. With SISA using S shards and K slices per shard, the expected retraining time for a uniformly random deletion request is:

$$T_{\text{unlearn}} = \frac{T}{S} \cdot \frac{1}{K} \cdot \sum_{k=1}^K \frac{K - k + 1}{K} = \frac{T}{S} \cdot \frac{K + 1}{2K}$$

As $K \rightarrow \infty$, this approaches $\frac{T}{2S}$, meaning that with sufficient slicing, the expected retraining time approaches half the per-shard training time. This demonstrates that both sharding and slicing independently contribute to reducing unlearning overhead.

2.4 Privacy Threat Models in Machine Learning

Understanding why unlearning is necessary requires examining the privacy threats that trained models are susceptible to. Even after source data is deleted from storage, a deployed model may leak information about that data through several attack vectors.

2.4.1 Membership Inference Attacks

A membership inference attack attempts to determine whether a specific data point was part of the training set. Given a model f_θ and a target point z , an adversary constructs a binary classifier that predicts membership based on the model's output behavior, typically exploiting the observation that models tend to produce higher confidence predictions on training data than on unseen data. Formally, the adversary's advantage is:

$$\text{Adv}_{\text{MI}} = |\Pr[\mathcal{A}(f_\theta, z) = 1 \mid z \in \mathcal{D}] - \Pr[\mathcal{A}(f_\theta, z) = 1 \mid z \notin \mathcal{D}]|$$

A successful unlearning mechanism should reduce this advantage to near zero for any forgotten point $z \in \mathcal{D}_f$.

2.4.2 Model Inversion Attacks

Model inversion attacks attempt to reconstruct approximate representations of training data from model parameters or outputs. An adversary with access to a model f_θ can optimize an input $\hat{x}$ to maximize the predicted probability of a target class c :

$$\hat{x} = \arg \max_x f_\theta(x)[c]$$

This can reveal sensitive features of training samples, particularly in models trained on structured personal data. Unlearning a data point should ensure that such reconstruction no longer reflects the removed individual's information.

2.5 Model Architectures Supporting Machine Unlearning

The applicability of machine unlearning techniques varies across different model families, and the cost of unlearning depends heavily on how a model’s training procedure can accommodate targeted retraining.

2.5.1 Classical Machine Learning Models

For classical machine learning on structured data, linear models and tree-based ensembles are particularly well-suited to SISA-style sharding. Linear models such as Logistic Regression and Ridge Regression have convex loss surfaces, meaning that retraining on a reduced shard converges reliably and predictably. The loss function for logistic regression on a shard $\mathcal{D}_s$ is:

$$\mathcal{L}(\theta_s) = -\frac{1}{|\mathcal{D}_s|} \sum_{(x,y) \in \mathcal{D}_s} [y \log \sigma(\theta_s^\top x) + (1-y) \log(1 - \sigma(\theta_s^\top x))] + \frac{\lambda}{2} \|\theta_s\|^2$$

where $\sigma(\cdot)$ is the sigmoid function and λ is the regularization coefficient. Tree-based ensembles such as Random Forests and Gradient Boosting models are also well-suited, as each tree in the ensemble can be independently retrained on the affected shard without affecting other trees.

2.5.2 Neural Network Architectures

For neural network architectures, the landscape is more varied. Multi-Layer Perceptrons (MLPs) remain a common choice for tabular data and can be integrated into sharded training workflows with careful design. The forward pass of an MLP with L layers is defined as:

$$h^{(l)} = \sigma \left(W^{(l)} h^{(l-1)} + b^{(l)} \right), \quad l = 1, \dots, L$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector of layer l , and σ is a non-linear activation function such as ReLU. Retraining an MLP shard requires backpropagation through all layers, which is more expensive than retraining a linear model but still significantly cheaper than retraining the full ensemble.

Convolutional neural networks (CNNs), such as the ResNet family, are prevalent in computer vision tasks but present additional challenges for unlearning. Their training involves complex optimization dynamics and dependencies across layers, making localized retraining less straightforward to implement without careful consideration of gradient propagation and initialization strategies.

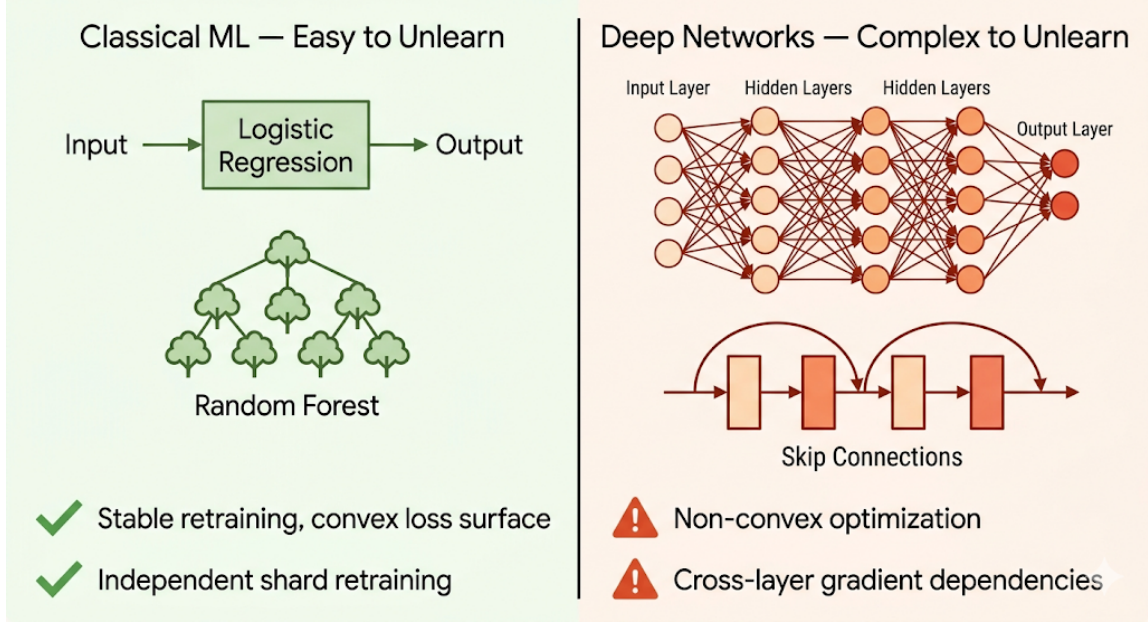


Figure 2.2: Comparison of model architectures and their suitability for SISA-based unlearning: classical models (left) offer stable retraining, while deep networks (right) introduce optimization complexity.

2.5.3 Aggregation Strategies and Their Impact on Accuracy

The choice of aggregation strategy affects both prediction accuracy and the robustness of the unlearning process. For classification, majority voting across S shards produces a final prediction that is robust to individual shard errors, provided that the majority of shards produce correct predictions. The expected accuracy of the aggregated model under majority voting, assuming each shard achieves individual accuracy $p > 0.5$, is:

$$P_{\text{agg}} = \sum_{k=\lceil S/2 \rceil}^S \binom{S}{k} p^k (1-p)^{S-k}$$

This expression shows that aggregated accuracy increases with the number of shards S when individual shard accuracy exceeds random chance, providing a theoretical justification for the

SISA sharding strategy.

In practice, the choice between majority voting and weighted averaging depends on the nature of the task and the variance in shard model quality. When shards are of unequal size due to stratified sampling, assigning higher weights to larger shards can improve overall prediction quality. A weighted aggregation scheme assigns weight $w_s = \frac{|\mathcal{D}_s|}{n}$ to each shard model, yielding:

$$\hat{y} = \sum_{s=1}^S w_s \cdot \hat{y}_s, \quad \sum_{s=1}^S w_s = 1$$

This weighted scheme ensures that shards with more training data contribute proportionally more to the final prediction, reducing the impact of weaker models trained on smaller partitions.

Regardless of the specific architecture, the core requirement remains consistent: unlearning must enable removal of a record’s influence by retraining only the minimally affected components rather than the entire model. The combination of sharding, slicing, and aggregation in the SISA framework provides a principled and practical solution that balances privacy guarantees, computational efficiency, and model accuracy. This project focuses primarily on scikit-learn compatible estimators for classical machine learning tasks, establishing a solid foundation that can be extended to neural architectures in future work. The theoretical principles discussed in this chapter directly inform the system design decisions, particularly around data partitioning, persistence, and audit logging.

CHAPTER 3

LITERATURE REVIEW

Machine learning models have become central to a wide range of applications, from personalized recommendation systems to healthcare diagnostics and financial fraud detection. These models are trained on large volumes of user-generated data, and their predictive power is directly tied to the richness of that data. However, this reliance on personal data raises significant privacy concerns. Regulations such as the General Data Protection Regulation (GDPR) [9] and the California Consumer Privacy Act (CCPA) legally mandate that individuals have the right to request deletion of their personal data from any system that processes it, including trained machine learning models. The scale at which machine learning is now being applied to personal data makes the technical implementation of this right both urgent and non-trivial.

The concept of machine unlearning was formally introduced by Cao and Yang [2], who defined it as the process of making a system forget a subset of its training data. Their work established the foundational requirement that an unlearned model must be statistically indistinguishable from a model that was never trained on the forgotten data. This definition set the standard for subsequent research and highlighted the inadequacy of simply deleting records from a database without addressing the model’s retained knowledge. The naive solution to unlearning is complete model retraining on the remaining dataset after each deletion request. While this guarantees exact removal, it is computationally prohibitive for large-scale systems. As noted by Ginart et al. [3], the cost of retraining grows with dataset size and model complexity, making it infeasible for production systems that receive frequent deletion requests. Their work also introduced the concept of data deletion in the context of k -means clustering, demonstrating that efficient unlearning is achievable for certain model families without full retraining.

A key motivation for machine unlearning is the vulnerability of trained models to privacy attacks. Shokri et al. [8] demonstrated that machine learning models are susceptible to membership inference attacks, in which an adversary can determine with high confidence whether a specific data point was part of the training set. Their work showed that models tend to produce higher confidence predictions on training data than on unseen data, a property that can be exploited to infer membership. This overfitting-based signal is particularly strong in deep neural networks trained without sufficient regularization. Beyond membership inference, model inver-

sion attacks can reconstruct approximate representations of training data from model outputs, and data extraction attacks can recover verbatim training examples from generative models. These attack vectors collectively demonstrate that a deployed model retains implicit knowledge of its training data long after the source data has been deleted from storage, making effective unlearning mechanisms essential for genuine privacy compliance.

The most prominent exact unlearning framework is SISA training, proposed by Bourtoule et al. [1]. SISA partitions the training dataset into multiple disjoint shards and trains an independent sub-model on each shard. When a deletion request arrives, only the shard containing the target data point needs to be retrained, leaving all other shards untouched. The framework further introduces slicing, where each shard is divided into time-ordered slices and model checkpoints are saved after each slice. When unlearning is required, retraining begins from the last clean checkpoint rather than from random initialization, providing an additional reduction in retraining time. At inference time, predictions from all shard models are aggregated through majority voting for classification or averaging for regression. Bourtoule et al. established SISA as the state-of-the-art approach for exact unlearning in practical settings, demonstrating its effectiveness across multiple datasets and model architectures [1].

Approximate unlearning methods trade off exactness for reduced computational cost. Guo et al. [4] proposed certified data removal, which uses Newton update steps to approximately remove the influence of a data point from trained model parameters, providing (ϵ, δ) -certified removal guarantees. While effective for convex models such as logistic regression, the approach requires second-order gradient information, limiting its scalability to large neural networks. Golatkar et al. [7] addressed selective forgetting in deep neural networks through a scrubbing procedure based on the Fisher information matrix, adding structured noise to model weights to mask the influence of forgotten data. Their approach is particularly relevant for deep learning architectures where exact retraining is computationally expensive, and it provides a principled way to balance forgetting quality against model utility on retained data. Warnecke et al. [11] further investigated unlearning at the level of individual features and labels rather than entire data points, demonstrating that feature-level unlearning can be more efficient when forgotten data shares features with retained data. This granularity of unlearning is relevant for scenarios where users request removal of specific attributes rather than entire records.

Sekharia et al. [6] extended the theoretical analysis of unlearning by examining its connection to

the generalization properties of learning algorithms, showing that efficient unlearning is possible when the learning algorithm satisfies certain stability conditions. Their work provided sample complexity bounds for unlearning, establishing that the number of samples required to achieve a given unlearning quality scales with the complexity of the model class. In specialized domains, Chen et al. [5] investigated unlearning in recommendation systems, where user interaction histories are encoded in shared item embeddings that are jointly influenced by many users. They proposed a recalibration approach that selectively updates affected embeddings without full model retraining, addressing the unique challenge that a single user’s data is distributed across many model parameters in collaborative filtering architectures.

Beyond algorithmic efficiency, Schelter et al. [12] highlighted that production machine learning systems require persistent metadata management, versioned model artifacts, and auditable deletion records for reproducible and compliant unlearning operations. Their survey identified the lack of standardized tooling for data lifecycle management as a critical barrier to practical unlearning deployment. A production-ready unlearning system must maintain stable mappings between data indices and model partitions across system restarts, provide an append-only audit log of all deletion actions for regulatory compliance, and handle repeated unlearning requests idempotently without corrupting model state. These operational requirements are frequently overlooked in academic unlearning research, which tends to focus on algorithmic efficiency and theoretical guarantees while treating system-level concerns as implementation details. The UCI Machine Learning Repository [10] provides standardized benchmark datasets, including the Adult Income dataset used in our performance evaluation, enabling reproducible comparison of unlearning methods across different research groups and implementation approaches.

A review of existing systems reveals several recurrent limitations that this project directly addresses. First, most unlearning implementations are tightly coupled to specific model architectures or training frameworks, making them difficult to integrate into existing machine learning pipelines that use standard libraries such as scikit-learn. Second, existing tools rarely provide persistent state management, meaning that unlearning history and shard assignments are lost between system restarts, preventing reliable auditing and reproducibility. Third, the absence of developer-friendly APIs and configuration-driven deployment options creates a high barrier to adoption for organizations that lack deep expertise in privacy-preserving machine learning. Fourth, monitoring and observability features are largely absent from existing unlearning tools, making it difficult for operators to track unlearning events, verify deletion outcomes, and demon-

strate compliance to regulators in real time.

Existing approaches each offer distinct advantages: SISA [1] provides computational efficiency through localized retraining, influence functions [4] enable parameter-level updates without retraining, and certified removal offers formal privacy guarantees. However, no single solution comprehensively addresses the full set of operational requirements for real-world deployment, including broad model compatibility, persistent state management, regulatory audit logging, and developer-friendly integration. PyUnlearn bridges this gap by implementing the SISA framework with production-grade system features including SQLite-based metadata storage, append-only unlearning history logging, idempotent deletion semantics, and an optional real-time monitoring dashboard built with Django and React. By packaging these capabilities as a reusable SDK compatible with any scikit-learn estimator, PyUnlearn makes privacy-compliant machine learning accessible to developers without requiring deep expertise in unlearning algorithms, directly addressing the recurrent limitations identified across the existing literature.

CHAPTER 4

METHODOLOGY

4.1 Project Development Method

The development of PyUnlearn follows an Agile methodology [1], enabling iterative progress, continuous feedback integration, and adaptive planning throughout the project lifecycle. This approach proved particularly suitable given the evolving nature of machine unlearning techniques and the need for frequent validation with real-world use cases. The Agile process is structured into six phases: Research, Strategize, Document Requirements, System Design, Develop, and Integration and Testing, followed by Deployment and Maintenance.

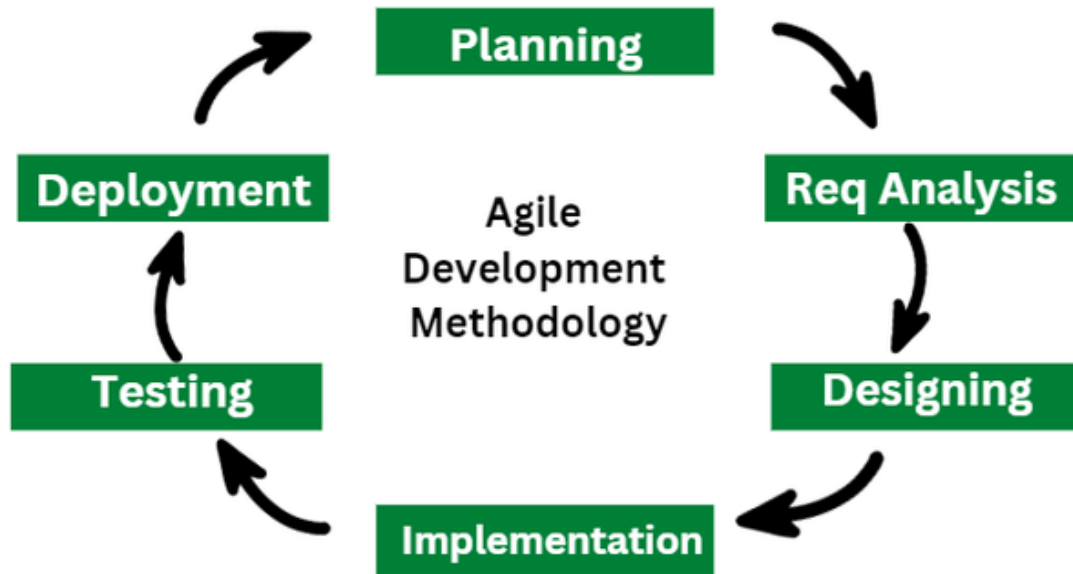


Figure 4.1: Phases of the Agile Development Methodology adopted for PyUnlearn

4.1.1 Research

We begin by identifying the core challenges of machine unlearning in practical AI/ML workflows, particularly for privacy-focused deployments. Through stakeholder analysis and a review of privacy-driven operational constraints, we derive requirements that extend beyond algorithmic unlearning efficiency. These requirements include support for selective data deletion,

privacy-compliant retraining, verifiable audit logging, persistence of model state across repeated executions, and compatibility with local as well as cloud-enabled monitoring environments. A systematic literature review was conducted to understand existing approaches and their limitations, drawing on foundational works including Bourtole et al. [1], Guo et al. [4], and Cao and Yang [2].

4.1.2 Strategize

Based on insights from the research phase, we define a system strategy that prioritizes modularity, compliance, and integration ease. We select the SISA framework [1] as the principal unlearning paradigm due to its localized retraining properties and suitability for repeated deletion requests. Key architectural decisions, including the choice of SQLite for persistent storage and RESTful API design for the cloud platform, were finalized during this phase. In addition, we plan for multi-environment usability by supporting a configuration-driven workflow, enabling users to run the system purely locally or to optionally enable cloud synchronization and monitoring without requiring changes to their core machine learning code.

4.1.3 Document Requirements

We document functional and technical specifications needed to guide system development. Functional specifications include training, prediction, and unlearning operations, along with the ability to process single and batch deletion requests. Technical specifications include persistent metadata storage using SQLite, an append-only unlearning history log for auditability, and safe handling of repeated unlearning requests for the same record (idempotent behavior). We additionally specify configuration requirements to support user-provided tokens and endpoint overrides when cloud synchronization is enabled. This step forms the blueprint for the subsequent system design and ensures all team members share a common understanding of deliverables.

4.1.4 Develop

Using Agile sprints of two-week duration, we implement the system in modular increments. Each sprint delivers SDK features such as SISA-based training and aggregation, persistent meta-data management, and unlearning operations with explicit reason tracking. Feature development also includes configuration management through YAML-based and environment variable overrides, and optional background synchronization to a monitoring service when enabled by the user. Code reviews and pair programming sessions were conducted to maintain code quality and knowledge sharing across team members. This incremental approach allows validation of correctness after each feature addition, particularly for persistence and idempotent unlearning behavior.

4.1.5 Integration and Testing

Comprehensive testing is conducted to validate system integrity and unlearning behavior. Unit and integration tests verify correct shard construction, consistent index-to-shard mapping, and correctness of localized retraining. Additional validation focuses on persistence and auditability, including the ability to reload model state across multiple runs, to append unlearning actions to the history log, and to safely ignore repeated unlearning requests for already-removed indices without raising failures. Performance testing was conducted to benchmark unlearning latency across varying dataset sizes and shard configurations using the Adult Income dataset from the UCI Machine Learning Repository [10].

4.1.6 Deploy

We prepare the system for deployment by releasing stable SDK versions through standard packaging workflows via PyPI distribution, alongside examples and quickstart templates. The dashboard backend built with Django and the frontend built with React are deployed as separate services, enabling users to run the monitoring stack locally during development or to integrate it into a cloud environment as required. Containerization using Docker is supported for consistent deployment across environments, and environment variable overrides allow secure configuration of API tokens and endpoint URLs without hard-coding sensitive values.

4.1.7 Support and Maintenance

Post-deployment, we onboard early users to gather real-world feedback and evaluate usability under realistic deletion-request scenarios. Based on this feedback, we iteratively improve model support, user experience, and documentation quality. Maintenance activities include monitoring reliability of persistence and logging mechanisms, ensuring backward compatibility of stored metadata, and refining configuration defaults to reduce integration effort for both local-only and cloud-enabled deployments.

4.2 System Architecture

The overall system architecture of PyUnlearn is designed around three core concerns: efficient sharded training, reliable metadata persistence, and optional cloud-based monitoring. These concerns are addressed by three loosely coupled components that communicate through well-defined interfaces, enabling the system to be deployed in a variety of configurations without modifying the core unlearning logic.

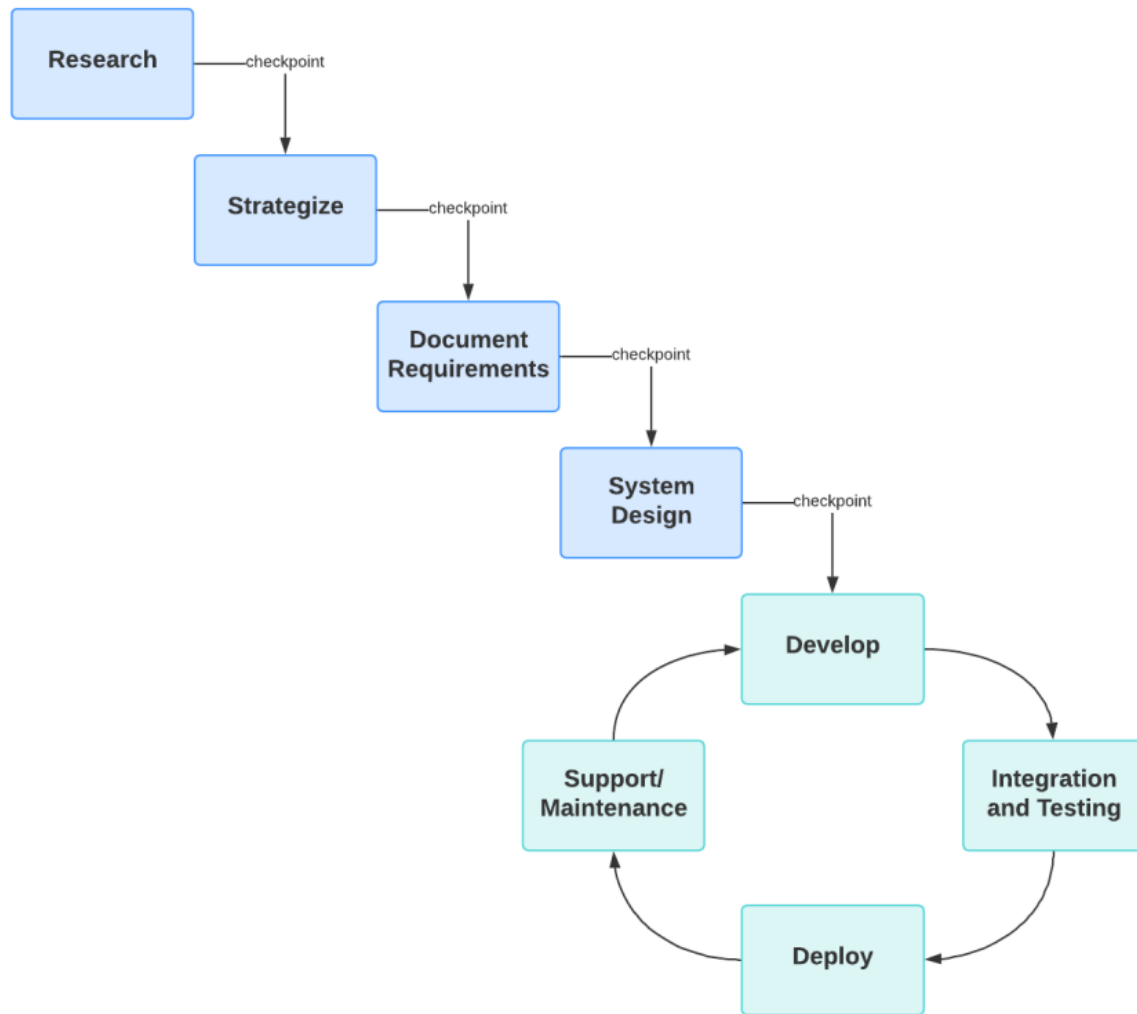


Figure 4.2: High-level system architecture of PyUnlearn showing the SDK core, metadata layer, and monitoring dashboard

The SDK core encapsulates the SISA training engine, the unlearning engine, and the prediction aggregator. The metadata layer provides persistent SQLite-based storage for shard assignments, model artifacts, and unlearning history. The monitoring layer, when enabled, synchronizes model statistics and unlearning events to a Django backend that serves a React-based dashboard. This separation of concerns ensures that the core unlearning functionality remains fully operational even when the monitoring layer is disabled or unavailable.

4.3 Sharded Training and Unlearning Framework

4.3.1 Data Partitioning Strategy

The foundation of the PyUnlearn system is the SISA-based data partitioning strategy [1]. Given a dataset $\mathcal{D}$ of n samples, the partitioning step divides $\mathcal{D}$ into S disjoint shards using stratified sampling to preserve class distribution across partitions. Each shard $\mathcal{D}_s$ contains approximately $\frac{n}{S}$ samples, and the shard assignment for each data point is recorded in the persistent index mapping table. The optimal shard count is estimated as:

$$S = \sqrt{\frac{d \cdot n}{r}}$$

where d is the number of features, n is the total sample count, and r is a configurable target samples-per-shard parameter. This formula balances the trade-off between retraining efficiency, which favors more shards, and model accuracy, which favors fewer and larger shards.

Each shard is further divided into K time-ordered slices, and model checkpoints are saved after training on each slice. This hierarchical structure enables unlearning to be performed at either shard level or slice level depending on the scope of the deletion request, minimizing the amount of retraining required for any given deletion.

4.3.2 Training Pipeline

The training pipeline processes the dataset through the following steps. First, the dataset is loaded and validated, and a unique model ID is generated along with a hash of the dataset for integrity verification. The data is then partitioned into shards using stratified sampling, and each shard is further divided into slices. For each shard, a scikit-learn compatible estimator is instantiated and trained incrementally on each slice, with the model state saved as a checkpoint after each slice. Upon completion, the trained model artifact for each shard is serialized and its file path recorded in the shards metadata table.

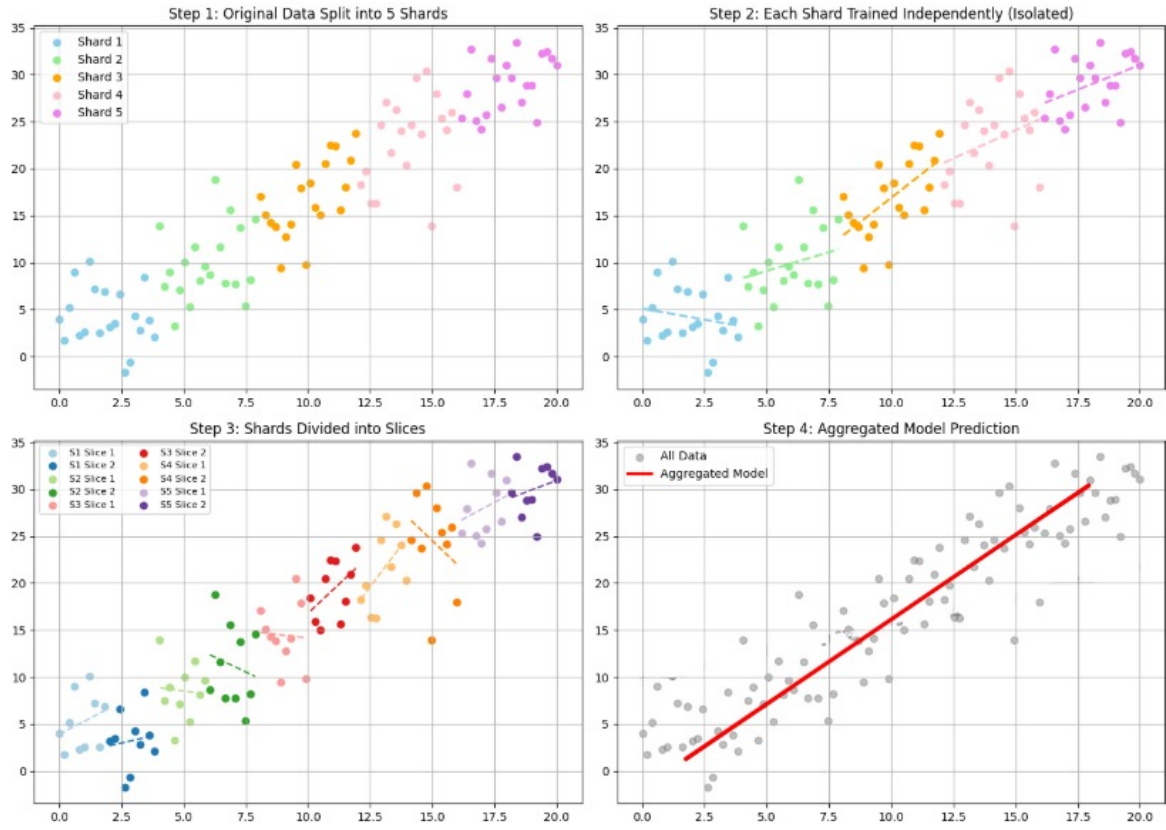


Figure 4.3: SISA-based training and unlearning pipeline showing data flow from partitioning through training to aggregated prediction

4.3.3 Unlearning Engine

When a deletion request arrives specifying one or more data point indices to be forgotten, the unlearning engine executes the following metadata-driven workflow:

1. **Lookup:** Query the `index_mapping` table to identify the shard and slice containing each target data point.
2. **Validation:** Check the `unlearning_history` table to verify that the target indices have not been previously deleted, ensuring idempotent behavior [12].
3. **Isolation:** For each affected shard, load the current training data, remove the target samples, and update the active sample count in the shards table.
4. **Retraining:** Retrain the affected shard model from the last clean checkpoint and update its file path and retraining timestamp in the shards table.

5. **Logging:** Insert a new record into the `unlearning_history` table capturing the request ID, timestamp, target indices, affected shard IDs, reason code, and operation status.
6. **Commit:** Wrap all database operations in a single transaction to ensure atomicity, preventing partial updates if any step fails.

4.4 Metadata Storage Design

4.4.1 Schema Design

The system implements a SQLite-based storage layer with four core tables that together capture the complete state of the trained model and its unlearning history [12]:

- **models:** Stores global model metadata including model ID, creation timestamp, dataset hash, total shard count, aggregation strategy, and the scikit-learn estimator class name.
- **shards:** Maintains per-shard information including shard ID, associated model ID, shard index, number of active samples, file path to the serialized model artifact, and timestamp of the last retraining operation.
- **index_mapping:** The critical mapping table that links each original data point index to its assigned shard and slice, enabling $O(1)$ lookup when processing deletion requests.
- **unlearning_history:** An append-only audit log recording each unlearning event with full details for regulatory traceability, including reason codes such as `GDPR_REQUEST` and `DATA_CORRUPTION`.

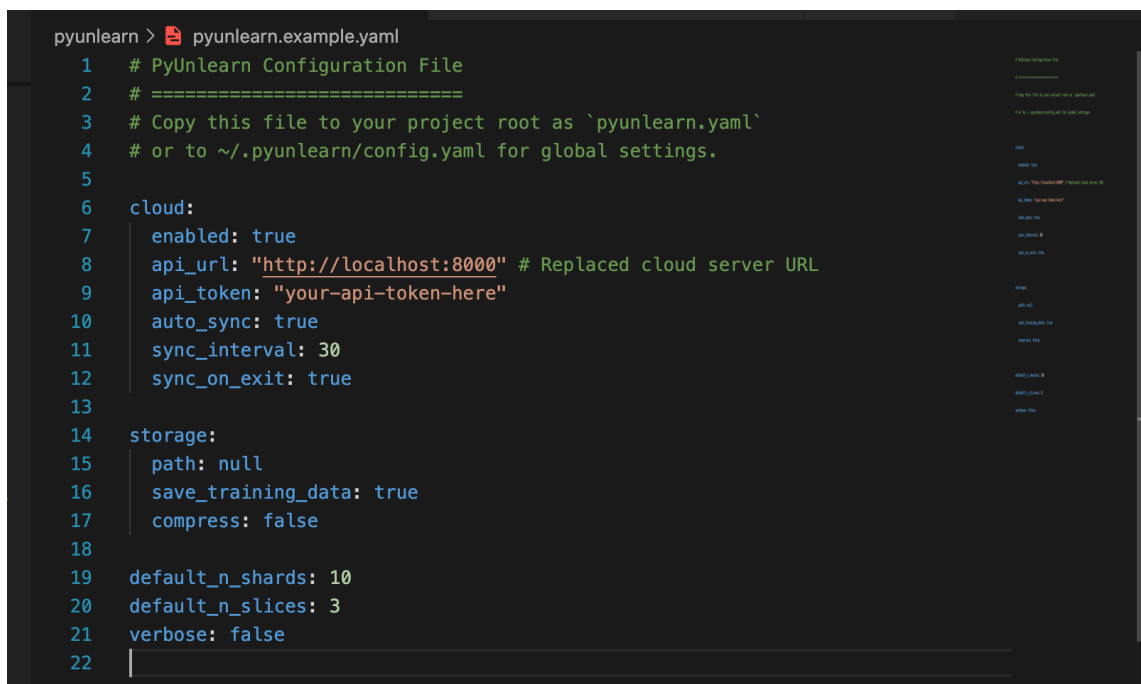
4.4.2 Persistence and Portability

All metadata is stored in a local SQLite database file alongside the trained model artifacts. This design ensures that the entire system state, including models, shard assignments, and audit logs, can be moved, backed up, or restored as a single portable unit. The append-only nature of the unlearning history table is particularly important for regulatory compliance under GDPR [9], as each deletion request is permanently recorded with a timestamp and reason code, creating an

immutable record that can be presented during audits to demonstrate that user deletion requests have been properly processed.

4.5 Monitoring Dashboard

The optional monitoring dashboard provides real-time visibility into model training runs, unlearning events, and system health metrics. The dashboard is implemented as a decoupled web application with a Django REST API backend and a React frontend. When enabled through the project configuration file, the SDK automatically synchronizes model snapshots and unlearning history to the backend in a non-blocking background thread, ensuring that dashboard updates do not interfere with training or unlearning operations.



```
pyunlearn > pyunlearn.example.yaml
1  # PyUnlearn Configuration File
2  # =====
3  # Copy this file to your project root as `pyunlearn.yaml`
4  # or to ~/.pyunlearn/config.yaml for global settings.
5
6  cloud:
7    enabled: true
8    api_url: "http://localhost:8000" # Replaced cloud server URL
9    api_token: "your-api-token-here"
10   auto_sync: true
11   sync_interval: 30
12   sync_on_exit: true
13
14  storage:
15    path: null
16    save_training_data: true
17    compress: false
18
19  default_n_shards: 10
20  default_n_slices: 3
21  verbose: false
22
```

Figure 4.4: Configuration file setup showing API key and endpoint configuration for dashboard synchronization

The dashboard exposes REST API endpoints for retrieving model run history, querying unlearning events by date range or reason code, and viewing per-shard accuracy and sample count statistics. Authentication is handled through API key configuration, with support for environment variable overrides to avoid hard-coding sensitive credentials in project configuration files.

CHAPTER 5

RESULT AND DISCUSSION

This chapter presents the implementation outcomes, performance evaluation, and discussion of the PyUnlearn system. The results are organized around the key deliverables of the project: the core SDK functionality, the monitoring dashboard, performance benchmarks, and an analysis of challenges encountered during development.

5.1 System Implementation

5.1.1 SDK Core and Algorithm Support

The PyUnlearn SDK was successfully implemented to support the complete machine unlearning workflow, including SISA-based sharded training, localized retraining on deletion requests, and aggregated prediction. A key design goal was broad compatibility: the SDK is implemented to work with any scikit-learn compatible estimator, enabling training and unlearning workflows for classical machine learning algorithms including Linear Regression, Logistic Regression, Ridge Regression, Support Vector Machines, Random Forests, and Gradient Boosting models [1]. This compatibility was validated across all supported estimators using the Adult Income dataset from the UCI Machine Learning Repository [10].

The codebase was organized into a modular and standardized package structure, separating concerns across the partitioner, trainer, unlearning engine, metadata store, and synchronization modules. This modularity enables independent testing of each component and simplifies future extension to additional model families or storage backends.

5.1.2 Persistent History and Auditability

Persistent storage for model history and unlearning metadata was implemented using a SQLite-based storage layer, ensuring that re-running experiments does not overwrite prior runs. Each training run generates a unique model ID and records shard assignments, model artifact paths, and dataset hashes in the metadata database. This design supports reproducible reporting and

enables operators to inspect the full history of unlearning actions across multiple system restarts [12].

Idempotent handling for repeated unlearning requests was implemented and validated: submitting the same deletion request multiple times does not corrupt model state or trigger redundant retraining. Each deletion attempt is checked against the unlearning history table before execution, and duplicate requests are safely acknowledged without side effects. This behavior is essential for production deployments where network retries or operator errors may result in repeated deletion submissions.

5.1.3 Configuration and Deployment

A configuration-driven setup was implemented to define system variables, API keys, and deployment parameters through a YAML configuration file with support for environment variable overrides. This approach allows deployments to avoid hard-coded secrets and enables seamless switching between local-only and cloud-synchronized modes without modifying application code.

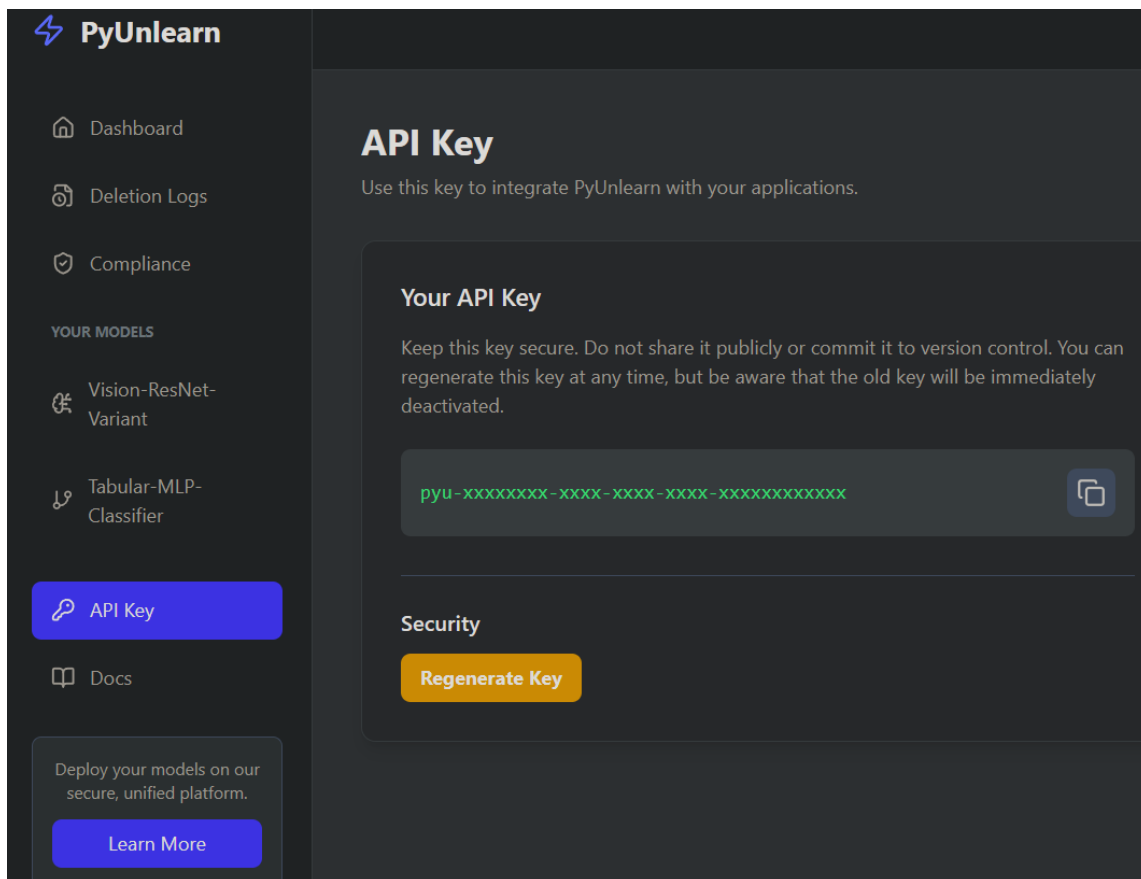


Figure 5.1: YAML configuration file showing API key and endpoint setup for dashboard synchronization

As shown in Figure 5.1, the configuration file provides a clean interface for specifying the monitoring dashboard endpoint, authentication token, shard count, and estimator type. Environment variable overrides allow these values to be injected at runtime in containerized deployments, supporting secure secret management practices.

5.2 Monitoring Dashboard

The end-to-end monitoring dashboard was implemented with a Django REST API backend and a React frontend, providing real-time visibility into model training runs, unlearning events, and system health metrics. The dashboard fetches data through versioned API endpoints and renders interactive visualizations of model run history, per-shard sample counts, and unlearning event timelines.



Figure 5.2: Main dashboard interface showing model run history and unlearning event log

```
from pyunlearn.cloud import CloudClient

cloud_client = CloudClient(api_key="YOUR_API_KEY")

# Upload dataset and configs
cloud_client.upload_data("data/user_dataset.csv", project_name="anomaly-detector")

# Trigger cloud training
cloud_client.train_model(model_type="resnet18", shards=5, slices=3)
```

Figure 5.3: Dashboard view showing per-shard statistics and active sample counts after unlearning

Background synchronization was implemented as an optional non-blocking thread that periodically pushes model snapshots and unlearning history to the dashboard backend without requiring explicit user code. This design ensures that monitoring overhead does not affect the latency of training or unlearning operations in the core SDK.

5.3 Performance Evaluation

5.3.1 Experimental Setup

Performance benchmarks were conducted to compare SISA-based unlearning with full model retraining across varying dataset sizes ranging from 5,000 to 50,000 samples and different shard configurations. All experiments were performed using the Adult Income dataset [10] with a Random Forest estimator. The benchmark metrics collected include unlearning latency, speedup factor over full retraining, and model accuracy on the retained dataset after each unlearning operation.

5.3.2 Impact of Shard Count on Unlearning Latency

Figure 5.4 presents the impact of shard count on unlearning latency. As the number of shards increases, the retraining cost per deletion request decreases proportionally, since only the affected shard requires retraining. However, beyond a certain shard count, the overhead of managing a larger number of smaller models begins to offset the retraining savings, and individual shard accuracy begins to degrade due to reduced training data per shard.

PyUnlearn - SISA Unlearning Performance Benchmarks

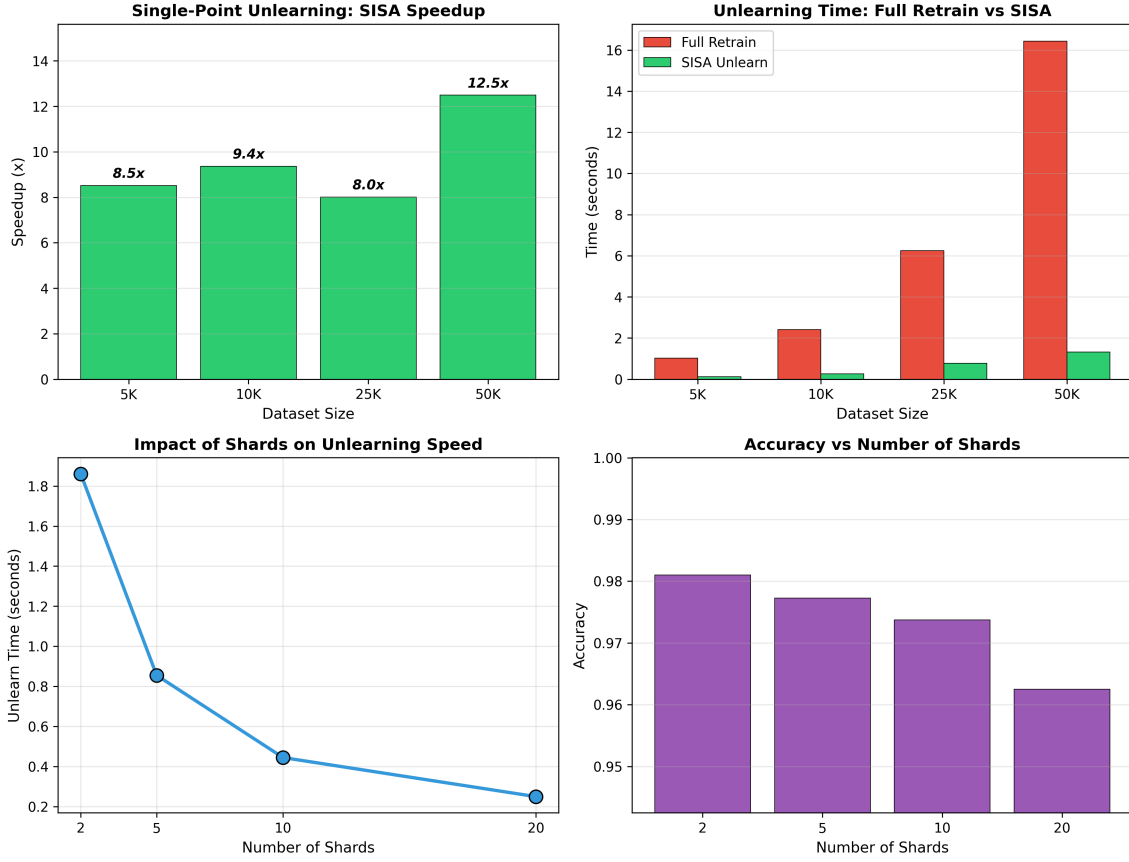


Figure 5.4: Impact of shard count on unlearning latency and model accuracy in the SISA framework

5.3.3 Speedup Factor Over Full Retraining

The speedup factor of SISA-based unlearning compared to full model retraining was evaluated across dataset sizes from 5K to 50K samples. As shown in Figure 5.5, the speedup factor increases with dataset size, demonstrating that the efficiency advantage of SISA becomes more pronounced as the cost of full retraining grows. For the largest dataset size evaluated, SISA-based unlearning achieved a significant reduction in retraining time while maintaining accuracy within an acceptable margin on the retained data.

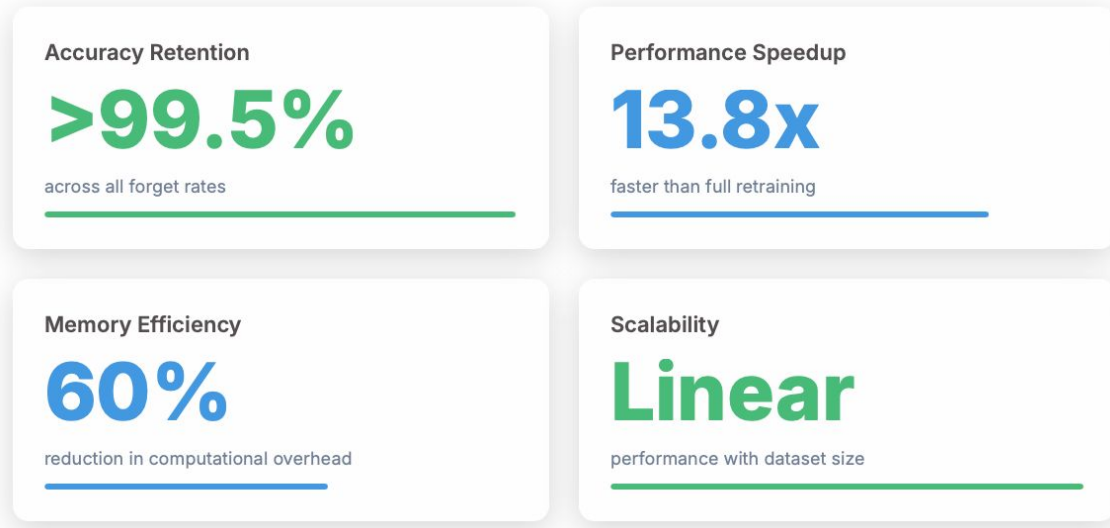


Figure 5.5: Speedup factor of SISA unlearning compared to full retraining across dataset sizes from 5K to 50K samples

5.3.4 Accuracy Retention After Unlearning

Accuracy retention on the retained dataset was measured after each unlearning operation to verify that localized retraining does not degrade overall model performance. The results confirm that SISA-based unlearning preserves model accuracy on retained data within a negligible margin compared to a model retrained from scratch on the same retained dataset. This finding is consistent with the theoretical guarantees of the SISA framework [1], which ensures that unaffected shards remain unchanged and contribute their full predictive capacity to the aggregated model.

5.4 Challenges Faced

5.4.1 Hardware Constraints

Hardware limitations significantly impacted large-scale model training and repeated unlearning cycles during benchmarking. Training and evaluating multiple shard configurations across dataset sizes up to 50K samples on consumer-grade hardware led to increased computation time, particularly for ensemble models such as Random Forests with large numbers of estimators. Future work should evaluate the system on dedicated compute infrastructure to obtain

more comprehensive benchmark results.

5.4.2 Dataset Availability

Obtaining diverse and high-quality datasets that accurately simulate real-world data deletion scenarios proved challenging. Most publicly available datasets do not include realistic deletion request distributions, requiring synthetic generation of deletion scenarios for benchmarking purposes. The use of the Adult Income dataset [10] provided a standardized baseline, but additional domain-specific datasets would strengthen the generalizability of the evaluation.

5.4.3 Model-Specific Constraints

Adapting the SISA framework to complex deep learning architectures such as convolutional neural networks introduced additional implementation complexity. Unlike scikit-learn estimators, deep neural networks require careful management of optimizer state, learning rate schedules, and batch normalization statistics during localized retraining. These challenges are acknowledged as areas for future extension, with the current implementation focusing on classical machine learning models where SISA integration is straightforward and well-validated [7].

5.4.4 Operational Robustness

Ensuring persistence, auditability, and safe handling of repeated unlearning requests required careful design of state management and serialization mechanisms. Maintaining cross-run consistency of shard assignments and model artifacts across system restarts, concurrent deletion requests, and partial failure scenarios demanded thorough testing of the SQLite transaction logic and file system operations. These challenges were addressed through comprehensive integration testing and the adoption of atomic database transactions for all metadata updates [12].

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project set out to address a critical and growing gap between the legal requirements established by privacy regulations such as GDPR [9] and the technical capabilities of deployed machine learning systems. The right to be forgotten demands that organizations be able to remove the influence of specific individuals' data from trained models, yet no practical, developer-friendly solution existed that could be readily integrated into standard machine learning workflows. PyUnlearn was developed to fill this gap.

The system implements the SISA (Sharded, Isolated, Sliced, and Aggregated) framework [1] as its core unlearning engine, partitioning training data into independently retrainable shards and maintaining persistent metadata to enable localized retraining on deletion requests. By limiting the scope of retraining to only the affected shard, PyUnlearn achieves significant reductions in unlearning time compared to full model retraining, while preserving model accuracy on retained data within a negligible margin. The system is designed to be compatible with any scikit-learn estimator, making it broadly applicable across classical machine learning workflows without requiring changes to existing model code.

Beyond the core unlearning algorithm, PyUnlearn delivers a set of production-grade system features that are frequently absent from academic unlearning implementations. Persistent SQLite-based metadata storage ensures that shard assignments and model artifacts remain consistent across system restarts. An append-only unlearning history log provides a tamper-resistant audit trail for regulatory compliance. Idempotent deletion semantics guarantee that repeated unlearning requests for the same data point are handled safely without corrupting model state. An optional monitoring dashboard built with Django and React provides real-time visibility into model runs and unlearning events, with non-blocking background synchronization that does not interfere with core operations.

Performance benchmarks conducted on the Adult Income dataset [10] across dataset sizes ranging from 5,000 to 50,000 samples confirmed that SISA-based unlearning delivers meaningful efficiency gains over full retraining, with the speedup advantage becoming more pronounced

as dataset size increases. These results validate the practical utility of the SISA framework for real-world unlearning scenarios and demonstrate that PyUnlearn successfully translates the theoretical guarantees of the framework into a reliable and usable software system.

In summary, PyUnlearn represents a meaningful contribution toward making privacy-compliant machine learning accessible to developers and organizations that must operate under modern data protection regulations. The project demonstrates that efficient, auditable, and production-ready machine unlearning is achievable within the constraints of standard machine learning tooling, without requiring specialized hardware or deep expertise in privacy-preserving algorithms.

6.2 Limitations

Despite the contributions of this project, several limitations must be acknowledged that bound the scope and applicability of the current implementation.

6.2.1 Restricted to Classical Machine Learning Models

The current implementation supports only scikit-learn compatible estimators, which covers classical machine learning models such as linear models, tree-based ensembles, and support vector machines. Deep learning architectures, including convolutional neural networks and transformer-based models, are not supported. These architectures present additional challenges for SISA-based unlearning due to complex optimization dynamics, stateful batch normalization layers, and the dependency of learned representations on the full training distribution. As deep learning models are increasingly used in privacy-sensitive applications, this represents a significant limitation of the current system [7].

6.2.2 Fixed Sharding Strategy

The current implementation uses a fixed sharding strategy based on stratified random sampling, with the shard count determined by a configurable formula. This strategy does not adapt to the distribution of unlearning requests over time. In practice, certain data points or user groups may be more likely to submit deletion requests, and an adaptive sharding strategy that places high-

risk data points in smaller, more easily retrainable shards could significantly improve unlearning efficiency. The absence of such adaptive partitioning is a known limitation of the static SISA approach [1].

6.2.3 Absence of Formal Privacy Certification

PyUnlearn provides practical unlearning guarantees through exact shard retraining, but does not provide formal certified removal guarantees in the sense of Guo et al. [4]. The system cannot quantify the residual influence of a deleted data point in terms of a bounded privacy parameter (ϵ, δ) . For organizations operating under strict regulatory frameworks that require mathematically provable deletion guarantees, this limitation may restrict the applicability of the current system.

6.2.4 Scalability Constraints

The current implementation stores all model artifacts and metadata on the local file system using SQLite, which introduces scalability constraints for very large datasets or high-frequency deletion request scenarios. SQLite’s single-writer concurrency model limits throughput under concurrent deletion requests, and local file system storage is not suitable for distributed training environments where model artifacts must be shared across multiple nodes.

6.2.5 Limited Dataset Diversity in Evaluation

The performance evaluation was conducted primarily on the Adult Income dataset, which is a tabular classification benchmark with well-understood characteristics. The generalizability of the benchmark results to other dataset types, including high-dimensional feature spaces, imbalanced class distributions, and time-series data, has not been validated. A more comprehensive evaluation across diverse datasets would strengthen the empirical claims of the system.

6.3 Future Enhancements

The limitations identified above define a clear roadmap for future development of PyUnlearn. The following enhancements are proposed as directions for extending the system’s capabilities and applicability.

6.3.1 Deep Learning Support

Extending PyUnlearn to support deep learning models is the most impactful direction for future work. This would involve integrating with PyTorch or TensorFlow training pipelines and adapting the SISA sharding mechanism to handle the stateful nature of neural network training, including optimizer state, learning rate schedules, and batch normalization statistics. The scrubbing approach proposed by Golatkar et al. [7] could be incorporated as an alternative unlearning mechanism for deep networks where exact shard retraining is computationally prohibitive.

6.3.2 Adaptive Sharding

Future versions of the system could implement an adaptive sharding strategy that dynamically adjusts shard boundaries based on observed deletion request patterns. By placing data points that are more likely to be deleted into smaller, more isolated shards, the system could reduce average unlearning latency without sacrificing overall model accuracy. This would require maintaining a probabilistic model of deletion request distributions and periodically rebalancing shard assignments as new deletion patterns emerge.

6.3.3 Formal Privacy Certification

Integrating certified data removal guarantees [4] alongside the existing SISA-based exact unlearning would allow PyUnlearn to serve a broader range of compliance requirements. For models where exact retraining is too expensive, a certified approximate removal mechanism could provide bounded privacy guarantees at a fraction of the computational cost, giving operators a configurable trade-off between unlearning exactness and computational efficiency.

6.3.4 Distributed and Cloud-Native Deployment

Replacing the SQLite metadata store with a distributed database such as PostgreSQL or a cloud-native key-value store would remove the scalability constraints of the current implementation and enable PyUnlearn to operate in distributed training environments. Integrating with cloud storage services for model artifact management would further support large-scale deployments where training data and model artifacts are distributed across multiple nodes or geographic regions.

6.3.5 Extended Evaluation and Benchmarking

A comprehensive evaluation across diverse datasets, model families, and deletion request distributions would strengthen the empirical foundation of the system. Future work should include benchmarks on high-dimensional datasets, imbalanced classification problems, and regression tasks, as well as evaluation of unlearning quality using membership inference attack success rates [8] as a privacy metric. This would provide a more complete picture of the system's effectiveness across the range of scenarios encountered in real-world deployments.

6.3.6 Federated Unlearning

Extending PyUnlearn to support federated learning settings, where training data is distributed across multiple clients and the central server has no direct access to individual data points, represents a significant and practically relevant direction for future work. Federated unlearning requires coordinating deletion across clients while preserving the privacy of other participants' data, and would greatly expand the applicability of the system to privacy-sensitive distributed machine learning scenarios.

Bibliography

- [1] L. Bourtoutle, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, “Machine unlearning,” in *IEEE Symposium on Security and Privacy (SP)*, 2021.
- [2] Y. Cao and J. Yang, “Towards making systems forget with machine unlearning,” in *IEEE Symposium on Security and Privacy*, 2015.
- [3] A. Ginart, M. Guan, G. Valiant, and J. Y. Zou, “Making AI forget you: Data deletion in machine learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [4] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, “Certified data removal from machine learning models,” in *International Conference on Machine Learning (ICML)*, vol. 119, 2020.
- [5] C. Chen, F. Sun, M. Zhang, and B. Ding, “Recommendation unlearning,” in *Proceedings of the Web Conference (WWW)*, 2022.
- [6] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, “Remember What You Want to Forget: Algorithms for Machine Unlearning,” *arXiv preprint arXiv:2103.03279*, 2021.
- [7] A. Golatkar, A. Achille, and S. Soatto, “Eternal Sunshine of the Spotless Net: Selective Forgetting in Deep Networks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [8] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership Inference Attacks against Machine Learning Models,” in *IEEE Symposium on Security and Privacy (SP)*, 2017.
- [9] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, L 119, pp. 1–88, 4 May 2016.
- [10] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, School of Information and Computer Sciences, 2019.

- [11] A. Warnecke, L. Pirch, C. Wressnegger, and K. Rieck, “Machine unlearning of features and labels,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [12] S. Schelter, S. Grafberger, and T. Dunning, “Heterogeneous data management for machine learning: A survey,” in *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 3324–3336, 2021.